

User & Role Access Matrix — AuthShield 360 v2

Single source of truth in code: `backend/portal.js` → `ROLE_ACCESS_MATRIX`,
enforced server-side by `portalGate(roles)` in `backend/routes.js`
(returns **401** without a session, **403** `PORTAL_RBAC_DENIED` on role mismatch).

Role inventory

Role	Who	What they get
ADMINISTRATOR	Security office	SOC console, forensics, benchmark, full portal, lists & control plane
TEACHER	Faculty	Portal scoped to own courses, class rosters
STUDENT	Learners	Portal scoped to self; denied admin-only modules
(prefix) <code>UNKNOWN</code>	unauthenticated / stale session	401 <code>SESSION_REQUIRED</code> / <code>SESSION_EXPIRED</code>

Access matrix (8 modules × 3 roles)

#	Module	Endpoint	STUDENT	TEACHER	ADMINISTRATOR
1	Courses	<code>GET /api/portal/courses</code>	✓ (enrolled)	✓ (own)	✓ (all)
2	Grades / Exam results	<code>GET /api/portal/grades</code>	✓ (own)	✓ (own courses)	✓ (all)
3	Assignments	<code>GET /api/portal/assignments</code>	✓ (enrolled)	✓ (own courses)	✓ (all)
4	Attendance	<code>GET /api/portal/attendance</code>	✓ (own)	✓ (own courses)	✓ (all)
5	Class rosters	<code>GET /api/portal/rosters</code>	✗ 403	✓ (own courses)	✓ (all)
6	User directory	<code>GET /api/portal/users</code>	✗ 403	✗ 403	✓ (all)
7	SOC monitoring	(SOC views in <code>view-soc</code>)	✗ 403	✗ 403	✓
8	Forensic investigations	<code>GET /api/forensics/*</code>	✗ 403	✗ 403	✓

Legend: ✓ allowed · ✗ denied (server-enforced 403) · *ownership scope in parentheses.

Server-side enforcement rules

- No session / expired / mismatched user:** `portalGate()` → **401** `{error: "SESSION_REQUIRED" | "SESSION_EXPIRED" | "SESSION_USER_MISMATCH" | "SESSION_UNKNOWN"}`.
- Role mismatch:** `portalGate(roles)` → **403** `{error: "PORTAL_RBAC_DENIED", role, requires}`.

- 3. **Data scoping:** TEACHER queries are injected with `WHERE c.teacher_username = ?`; STUDENT rows are self-scoped (`WHERE e.student_username = ?`); ADMIN sees everything.
- 4. Same gate is reused by the **admin-only control plane** (`adminGate` checks the in-memory active-SID set): `/api/lists`, `/api/blacklist`, `/api/whitelist`, `/api/factors`, `/api/factors/lockout`, `/api/devices`, `/api/security-policy`, `/api/arm-mode`, `/api/forensics/*`.
- 5. **SOC route lock:** SRS Scenario 3 from a STUDENT identity is consumed by RBAC before success — outcome `HIGH_RISK_BLOCK` with reason `UNAUTHORIZED_ROUTE` (audited + alerted).

Evidence (automated)

- `scripts/e2e-all.js` §11 — 14 assertions across the three roles (401, 403s, allow-lists, 8×3 matrix).
- `scripts/ui-smoke.js` — student sees Security Center only; teacher sees Portal; admin sees all tabs + matrix.
- Statuses verified on a fresh DB: **77/77 e2e**, **28/28 UI**, **17/17 smoke**, **18/18 restart**.

Authoritative code pointers

Concern	Location
Matrix definition	<code>backend/portal.js</code> <code>ROLE_ACCESS_MATRIX</code>
Session/role validation	<code>backend/portal.js</code> <code>validatePortalSession</code> / <code>requireRoles</code>
HTTP gating	<code>backend/routes.js</code> <code>portalGate(roles)</code> / <code>adminGate</code>
Data scoping queries	<code>backend/db.js</code> <code>courseRosters</code> , <code>examResultsFor</code> , <code>attendanceFor</code> , <code>assignmentsFor</code>
RBAC login block	<code>backend/auth.js</code> §9 (scenario 3 + STUDENT → <code>UNAUTHORIZED_ROUTE</code>)